

Operating System #3. Synchronization, Race condition, and Lock.

1. Multiple Execution Flows.

"Modern systems run many execution flows"

<Examples>

- a browser downloads data while rendering a page.
- a webserver handles many users
- a database processes many transactions
- an OS handles keyboards, disk, network, and timer events.

<Read problems>

- Multiple execution flows may access the same data or resource at the same time.
- If all flows only read the data, it's usually safe.
- If at least one flow modifies the data, the result may become incorrect.

2. Shared Resource

2.1. Definition: resource that can be accessed by more than one process, thread, or execution flow.

<examples>

global variable, object field, heap data, file, database row, memory buffer, queue, socket, printer, counter variable, bank account balance.

2.2. Safe Sharing.

"Read - only Sharing is ~~not~~ usually safe."

<Example>

Thread A reads config value

Thread B read config value.

If no thread changes the value, the result is predictable"

2.3. Dangerous Sharing.

"Shared mutable state." → "the value affects program behavior."

↓
"more than one execution flow can" access.

↻
"the value can change"

↳ "Synchronization is mainly needed because of shared mutable state"

3. Interleaving.

3.1. Def: Instructions from multiple execution flows are mixed in different orders.

<Example>.

When you code like this; "counter++;" in C language.

It can be divided into small steps

- (① read counter from memory.
- ② add 1
- ③ write the result back to memory.)

↳ The OS scheduler may switch threads between these small steps.

3.2. Lost Update Example

<Initial values>

counter = 0;

<2 threads both run>

counter++; (twice)

<possible order>

- Thread A reads counter=0
 - Thread B reads counter=0
 - Thread A adds 1
 - Thread B adds 1
 - Thread A writes 1
 - Thread B writes 1
- Annotations: Brackets on the right group the reads to 0 (2), the adds to 1 (2), and the writes to 1 (2). An arrow points from the second 'add 1' to the first 'write 1'.

(Expected result = 2.
Actual result = 1.)

Why? $\Rightarrow$ one update disappeared

3.3. Why Interleaving matters.

"different interleavings can produce different results"

- the program may work sometimes,
- the program may fail sometimes
- the bug disappear during debugging.
- the result depends on timing.

<key point>

Concurrent bugs are difficult because

the wrong execution order may happen only sometimes.

4. Race Condition

4.1. Def: a situation where the correctness of a program depends on the timing or order of execution.

<usually happens when>

- multiple execution flows access shared data.
- at least one flow modifies the data.
- the operations are not properly synchronized.
- different execution orders can produce different results.

4.2. Simple Example.

```
int counter = 0;  
void increment() {  
    counter++;  
}
```

If 2 threads call increment() at the same time, the final value may be wrong.

4.3. Often happen in.

queues, caches, files, shared object, database record, user session data, payment system, multithreaded servers.

5. Critical Section.

5.1. Def

"a part of code that accesses shared resources and must not be executed by multiple execution flows at the same time."

In Short, "dangerous shared-data code that needs protection"

5.1. Example.

```
lock(mutex);  
counter++;  
unlock(mutex);
```

only one thread should enter this part at a time.

6. Critical Section Requirement

6.1. Mutual Exclusion.

"only one execution flow can be inside the critical section at a time"

ex) If thread A is using shared data, Thread B must wait."

6.2. Progress

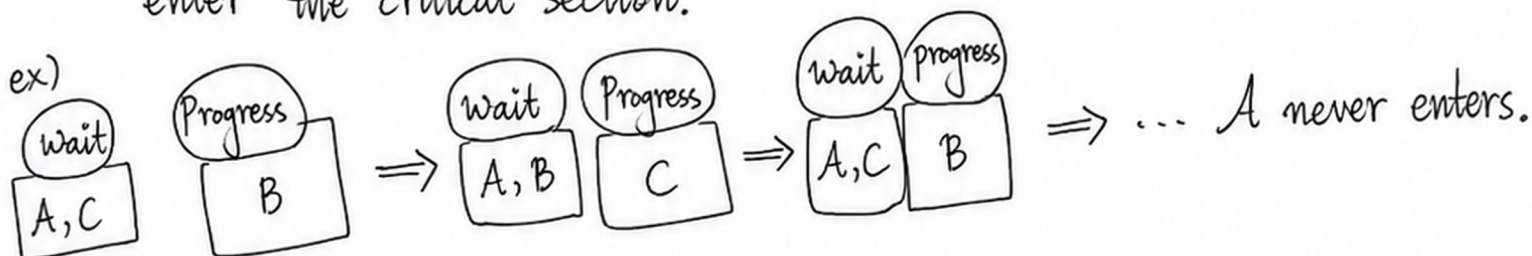
"Nothing is inside the critical section, one waiting thread should be able to enter."

<Bad case>

No thread is inside the critical section, but all threads wait forever.

6.3. Bounded Waiting

"a thread shouldn't wait forever while other threads repeatedly enter the critical section."



7. Atomic Operation.

7.1 Def

"an operation that is completed as one indivisual unit"

→ all or nothing If an operation is atomic, others can't observe its half-finished state.

7.2. Non-Atomic Example.

```
counter++;
```

↳ several steps

- ① read counter's value.
- ② add 1.
- ③ write counter.

7.3. Atomicity and Locks.

Locks are used to make a section of code behave like one protected unit

```
lock(mutex);  
counter++;  
unlock(mutex);
```

⇒ from other threads' viewpoint, only one thread can execute that protected part at a time.

↳ lock gives controlled access.

8. Synchronization

8.1. Def.

The technique of coordinating multiple execution flows, so they can use shared resources.

<needed when>

- data is shared.
- data can change.
- multiple execution flow can access it.
- incorrect interleaving can break the result.

8.2. Synchronization protects...

- ① shared variables, objects, shared objects, queues, memory buffers.
- ② files
- ③ database records
- ④ device access.
- ⑤ kernel data structures.

8.3. Synchronization tries to protect...

- ① race condition
- ② lost updates,
- ③ corrupted data
- ④ inconsistent state
- ⑤ unsafe simultaneous access.

8.4 Synchronization Cost.

- waiting time.
- context switching.
- reduce parallelism.
- lock contention.
- deadlock risk.
- more complex code.

9. Lock.

9.1. Def : an access token for a critical section.
only the thread holding the lock may enter.

9.2. Basic Pattern

```
lock(mutex);  
// read or update shared state,  
unlock(mutex);
```

Flow: (acquire) → (use shared data) → (release.)

9.3. When to use it.

- shared data can change
- multiple threads may access it
- the operation has multiple steps, such as '+' update.

9.4. Release Rule.

```
lock(mutex);  
try {  
    // critical section  
} finally {  
    unlock;  
}  
}
```

The code that acquire a lock
must guarantee its release.

10. Mutex.

- Def: mutual exclusion lock.
- It allows one owner at a time.
- Typical flow
 - Thread A acquires the mutex and enters
 - Thread B requests the same mutex and waits
 - Thread A releases it; thread B may enter.

11. When a Lock is unavailable.

11.1. Blocking Lock.

- Waiting for: Thread sleeps (CPU can run other work)
- Best use: Normal or longer waits

11.2. Spinlock

- Waiting for: Thread repeatedly checks (CPU stays busy)
- Best use: Very short wait only.

Long wait



block. Very short wait



spinning may be cheaper than
sleeping and waking

12. Lock Granularity.

"how much data or code one lock protects"

12.1 Coarse-grained

- one large lock.
- Simple, but more waiting

12.2. Fine-grained

- Several smaller locks.
- More concurrency, but more complexity.

13. Lock Contention.

"it occurs when many thread compete for the same lock"

13.1 Main effect.

- more waiting and context switching.
- lower throughput and slower response time.
- multiple CPU cores become underused.

13.2. it can be reduced:

- keep critical sections short.
- move network/file I/O outside the lock.
- ~~reduce~~ reduce shared mutable state.
- use smaller locks only when needed.

14. Common Lock Mistakes.

- No lock; shared update can race.
- Unlock too early; another thread can change data before the update.
- Lock too late; The value can change after check.
- Forget to unlock; Other threads may wait forever.
- Lock too much; correct, but unnecessarily slow.
- Different Lock order; create deadlock.

15 Dead lock.

"2 or more threads are waiting for each other forever" [☆]

<Example>

- Thread A: hold X → waits for Y
- Thread B: hold Y → waits for X.

<Basic prevention>

- acquire multiple locks in one global order.
- avoid unnecessary nested locks
- keep lock scope short.
- use try-lock or timeout when appropriate

16. Synchronization and Scheduling

- The scheduler may interrupt a thread between read, modify, and write
- A waiting thread often moves from running to blocked, then back to ready.
- This is why a short source-code line may still need synchronization.

17. Quick Comparison Summary

- Race Condition: Result depends on execution timing.
- Critical section: Shared state that needs protection
- Lock: General access control mechanism.
- Mutex: one owner mutual exclusion lock.
- Blocking: Waiting thread sleeps.